



OPERATING SYSTEM

LAB 02 | SHELL SCRIPT

Learn how to write shell script to automate your task in Linux OS.



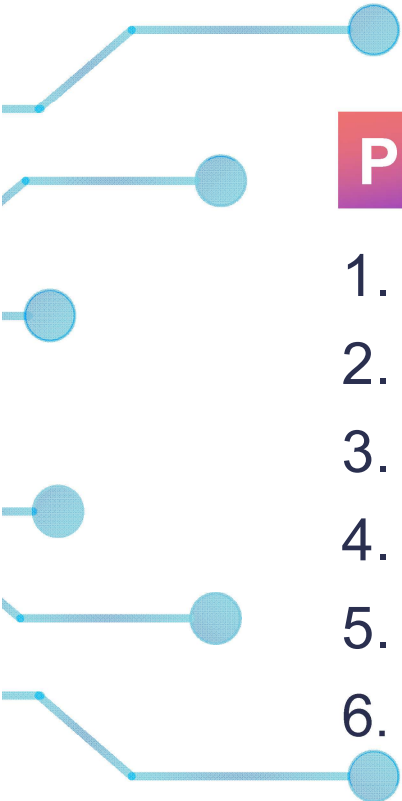
OUTCOMES

After this LAB, students will be able to:

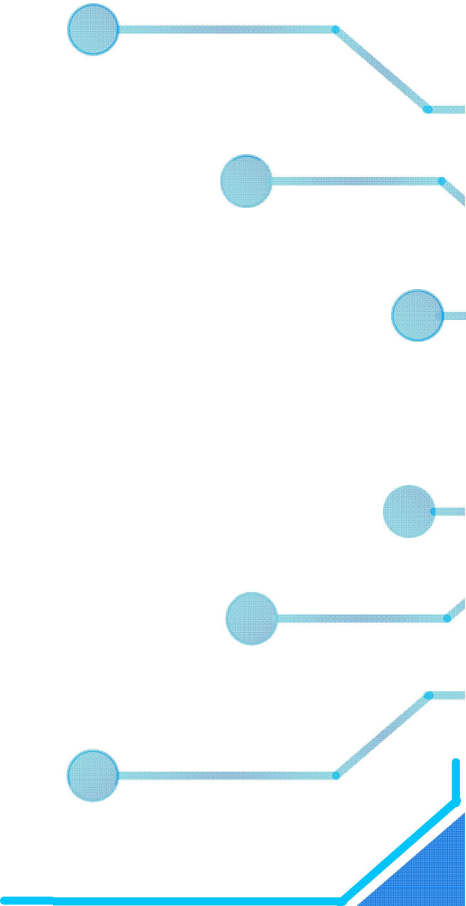
1. Clarify the differences of command types
2. Use shell's metacharacter in command
3. Describe the use of environment variables and parameter variables
4. Automate tasks in OS using script



CONTENS



PRACTICE

1. Commands in Linux
 2. Execute an External script
 3. Shell variables
 4. Shell's metacharacter
 5. Environment variables
 6. Parameter variables
- 



COMMANDS IN LINUX

01.



1. Commands in Linux

How does a command actually work?

Try

```
$ which cat  
/bin/cat
```

Let's check

```
$ ls /bin/
```

*Can you find the file named **cat**?*

↑ *It means the **cat** file is located in **/bin/***

→ **A command is actually a file**



1. Commands in Linux

Types of shell's command

Built-in Commands:

These commands are built into the shell itself. Examples include `cd`, `echo`, `pwd`, `exit`, `set`, and `unset`.

Aliased Commands:

These are shortcuts for longer commands or command sequences. They are created using the `alias` command. For example, `alias ll='ls -l'`



1. Commands in Linux

Types of shell's command

Shell Functions:

These are user-defined commands that are written within the shell and can be called like any other command.

Defining the function

```
greet(){  
    echo "Hello, $1! Welcome to Linux shell."  
}
```

Using the function

```
greet Lucas  
Hello, Lucas! Welcome to Linux shell.
```



1. Commands in Linux

Types of shell's command

Executable Programs:

These are standalone programs or scripts that are located in directories listed in the PATH environment variable. Examples include ls, grep, awk, sed, find, and vim

```
which ls  
/bin/ls
```

```
which grep  
/usr/bin/grep
```

```
which find  
/usr/bin/find
```

```
which vim  
/usr/bin/vim
```




1. Commands in Linux

Types of shell's command

Executable Programs:

These are standalone programs or scripts that are located in directories listed in the PATH environment variable. Examples include ls, grep, awk, sed, find, and vim

```
echo $PATH
```

```
/opt/anaconda3/bin:/opt/anaconda3/condabin:/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/Library/Apple/usr/bin
```



1. Commands in Linux

Types of shell's command

External Shell Scripts:

These are scripts written in the shell scripting language and executed by the shell. They often have a .sh extension and are executed by specifying their path

Control Commands:

These include commands used to control the flow of execution in shell scripts, such as if, else, for, while, case, and select



1. Commands in Linux

Types of shell's command

Job Control Commands :

These commands manage background and foreground processes, including `bg`, `fg`, `jobs`, `kill`, `wait`, `nohup`, and `disown`

Environment Variable Commands:

These commands manage environment variables, such as `export`, `env`, `set`, and `unset`



EXECUTE AN EXTERNAL SCRIPT

02.



2. Execute an external shell script

In your machine, create file *findString.sh* as below:

Sample: findString.sh

```
#!/bin/sh

# findString.sh
# This script will find in current directory
# all files contain 'main( )'.
# Then print the content of that file to terminal

for file in *; do
    if grep -l 'main( )' $file; then
        more $file
    fi
done
exit 0
```

This **is not** a comment, it tells OS to use *sh* shell to execute this script

Comments start with #

Shell script



2. Execute an external shell script

There are 3 methods to run an external shell script/program in Linux using shell:

- Call shell and pass script as parameter
- Use script's or program's address
- Turn it into executable program



Try it yourself!

2.1. Method 1

Call shell and pass script as parameter

```
$ /bin/sh findString.sh
```

Address of script acts as parameter

Call sh shell to execute

Example

```
$ /bin/bash ~/Desktop/App/findString.sh
```

```
$ /bin/zsh ../findString.sh
```



Try it yourself!

2.2. Method 2

Use script's or program's address

```
$ ./findString.sh
```

Directly use script's address to execute it

Remember to set execute permission for the script

Example

```
$ ~/Desktop/App/findString.sh
```

```
$ ./findString.sh
```




Try it yourself!

2.3. Method 3

Turn it into executable program

\$ `findString.sh`

The script's name acts as a command

Remember to set execution permission for the script

However

```
⊗ (working) (base) lucas.tran@Hoang-Locs-Laptop TestServer % findString.sh
zsh: command not found: findString.sh
```

How can we solve it?



SHELL VARIABLES

03.



3. Shell variables

- By default, all variables are initialized as *string*.
- If you want to get value of variable, use **\$** before it.

Try it yourself!

```
$ hello= "Xin chao"
```

```
$ a=5
```

```
$ echo hello
```

```
$ b=10
```

```
$ echo $hello
```

```
$ echo $a+$b
```



3. Shell variables

- You can use **read** command to ask user input the value for a variable

Try it yourself!

```
$ echo "What's your name?"  
$ read name  
$ echo "Hello $name, nice to meet you!"
```



SHELL's METACHARACTER

4.1. Redirection character

04.



4.1. Redirection characters

Classwork: 01

Try all redirection characters and explain its usage?



4.1. Redirection characters

	Name	Usage	Example
>	Standard Output Redirection	Redirects the standard output (stdout) of a command to a file, overwriting the file if it already exists	ls > filelist.txt
>>	Append Standard Output	Appends the standard output of a command to the end of a file, creating the file if it doesn't exist	echo "New entry" >> filelist.txt



4.1. Redirection characters

	Name	Usage	Example
<	Standard Input Redirection	Redirects a file as the standard input (stdin) for a command	<code>sort < unsorted_list.txt</code>
<<	Here Document	Allows you to specify a multi-line string as input to a command until a delimiter is encountered	<code>\$ cat << EOF</code> <code>>heredoc Line 1</code> <code>>heredoc Line 2</code> <code>>heredoc EOF</code>



4.1. Redirection characters

	Name	Usage	Example
2>	Standard Error Redirection	Redirects the standard error (stderr) of a command to a file, overwriting the file if it already exists	<code>grep "something" somefile 2> errors.txt</code>
2>>	Append Standard Error	Appends the standard error of a command to the end of a file, creating the file if it doesn't exist	<code>grep "something" somefile 2>> errors.txt</code>



4.1. Redirection characters

	Name	Usage	Example
&>	Redirect Both Standard Output and Standard Error	Redirects both stdout and stderr to a file, overwriting the file if it already exists	command &> output.txt
&>>	Append Both Standard Output and Standard Error	Appends both stdout and stderr to the end of a file, creating the file if it doesn't exist	command &>> output.txt



SHELL's METACHARACTER

4.2. Process control character

04.



4.2. Process control character

1. & - Ampersand

- Append **&** at the end of a command to run it in the background

Example

```
$ sleep 10  
$ sleep 10 &
```



4.2. Process control character

1. & - Ampersand

Sample: count.sh

```
#!/bin/bash
# count.sh

# Check if an argument is provided
if [ -z "$1" ]; then
    echo "Please provide a number as an argument."
    exit 1
fi

# Iterate from 0 to the provided argument
for ((i=0; i<=$1; i++))
do
    echo "Iteration: $i" >> result.txt
    sleep 1
done
```

The beside script will receive a number from command line and then count from 0 to that number. The result is redirected to a file named **result.txt**.

```
$ ./count.sh 10
```

```
$ ./count.sh 10 &
```

Are there any differences?



4.2. Process control character

Try it yourself!

1. & - Ampersand

- You can use the command **jobs** to list process running in background

```
(working) (base) lucas.tran@Hoang-Locs-Laptop TestServer % ./count.sh 10 &
[1] 74605 PID
(working) (base) lucas.tran@Hoang-Locs-Laptop TestServer % jobs
[1] + running ./count.sh 10 Process running in background
(working) (base) lucas.tran@Hoang-Locs-Laptop TestServer %
[1] + done ./count.sh 10 Background process finished
```



4.2. Process control character

Try it yourself!

1. & - Ampersand

- You can also use the command **ps** to check running process

```
(working) (base) lucas.tran@Hoang-Locs-Laptop TestServer % ./count.sh 10 &
[1] 74860
(working) (base) lucas.tran@Hoang-Locs-Laptop TestServer % ps
  PID TTY          TIME CMD
 70208 ttys001    0:00.41 /bin/zsh -il
 74860 ttys001    0:00.00 /bin/bash ./count.sh 10
 74867 ttys001    0:00.00 sleep 1
```

Process is running (in bg)



4.2. Process control character

Try it yourself!

1. & - Ampersand

- If you want to bring the process back to foreground, use the command **fg**

```
(working) (base) 192:TestServer lucas.tran$ ./count.sh 10 &  
[1] 76307
```

```
(working) (base) 192:TestServer lucas.tran$ fg 1  
./count.sh 10
```

1 is the order of background task



4.2. Process control character

2. (;) – Parentheses

- Use to group commands
- Separated by semicolon ;

Try it yourself!

Example

```
$ (date ; whoami) > system.status  
$ cat system.status
```



4.2. Process control character

3a. `command` – Backquotes

3b. \$(command)

- Used for command **substitution**, which allows **the output** of a command to be **used as an argument** in another command.
- This can be useful for dynamically generating arguments or configurations based on the output of other commands

Try it yourself!

Example

```
$ current_date=`date`
```

```
$ echo "Today is $current_date"
```



4.2. Process control character

3a. `command` – Backquotes

3b. \$(command)

- Modern alternative of `command` is `$(command)`
- This approach is more readable and easier to nest if needed.

Try it yourself!

Example

```
$ current_date=$(date)
$ echo "Today is $current_date"
```



4.2. Process control character

3a. `command` – Backquotes

3b. \$(command)

Nesting commands with backquotes (less readable)

```
$ outer_output=`echo "Outer: `echo Inner`"`
```

```
$ echo $outer_output
```

Nesting commands with \$() (more readable)

```
$ outer_output=$(echo "Outer: $(echo Inner)")
```

```
$ echo $outer_output
```



4.2. Process control character

4. | - Pipelines How does it work?

1. Command Output Redirection:

- Normally, a command's output is displayed on the terminal (standard output, or *stdout*).
- With a pipeline, this output is **redirected** to another command.

2. Pipe Operator (|):

- The pipe operator (|) connects the **output** of one command to the **input** of the next.



4.2. Process control character

4. | - Pipelines Example

Simple use: *Filters the output to show only directories*

```
$ ls -l | grep "^d"
```

- `ls -l`: Lists files and directories in long format.
- `grep "^d"`: Find lines starting with d.



4.2. Process control character

4. | - Pipelines Example

Complex use:

```
$ ps aux | grep apache | awk ' {print $2}' | xargs kill -9
```

Classwork: 02

Explain the meaning of above command. Google if you need help.



4.2. Process control character

4. | - Pipelines Common commands used in pipelines

grep: Searches for patterns within text.

awk and **sed**: Text processing and transformation.

cut: Cuts out sections from each line of files.

sort: Sorts lines of text files.

uniq: Reports or filters out repeated lines.

wc: Counts lines, words, and characters.

xargs: Builds and executes command lines from standard input.



SHELL's METACHARACTER

4.3. Quoting

04.



4.3. Quoting

- Shell uses a set of special characters to enable or disable the special meanings of other characters
- When a special character is *quoted*, it loses its special meaning



4.3. Quoting

Try it yourself!

Two types of quotes

Single Quotes ‘ ’

- Preserves the literal value of each character within the quotes.
- Example: `echo 'text with $variable'` remains unchanged.

```
$ variable=10
```

Double Quotes “ ”

- Preserves the literal value of most characters within the quotes.
- Exceptions: \$, `, and \ retain their special meanings.
- Example: `echo "text with $variable"` expands the variable.



4.3. Quoting

Try it yourself!

Sample: variables.sh

```
#!/bin/sh
myvar="Hi there"

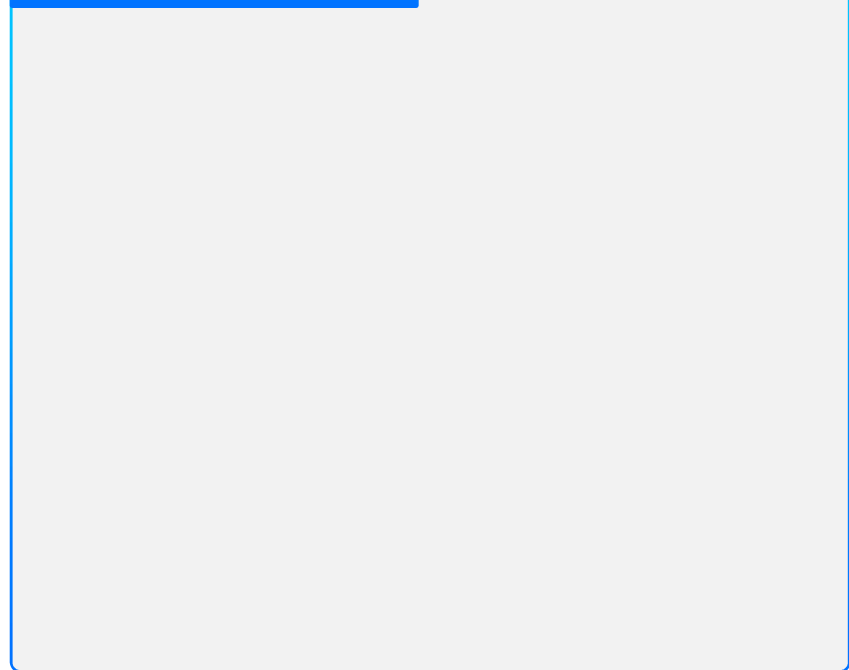
echo $myvar
echo "message : $myvar"
echo 'message : $myvar'
echo "messgae : \"$myvar"

echo Enter some text
read myvar

echo '$myvar' now equals $myvar

exit 0
```

Output:





ENVIRONMENT VARIABLES

05.



5. Environment variables

Env. variables	Meanings
\$HOME	The home directory of the current user
\$USER	The current logged-in user
\$PATH	A list of directories that the shell searches for executables
\$SHELL	The path to the current user's shell



5. Environment variables

Env. variables	Meanings
\$PWD	The current working directory
\$HOSTNAME	The name of the host
\$PS1	The primary prompt string
\$#	Number of command line parameters



PARAMETER VARIABLES

06.



6. Parameter variables

Param. variables	Meanings
\$1, \$2, \$3, ...	The position and value of cli params
\$*	Represents all the positional parameters passed to a script or function. It concatenates all the arguments into a single string, separated by the first character of the IFS
\$@	Same meaning as \$* but preserve each argument as a distinct word, even if they contain spaces



6. Parameter variables

Try it yourself!

Sample: try_variables.sh

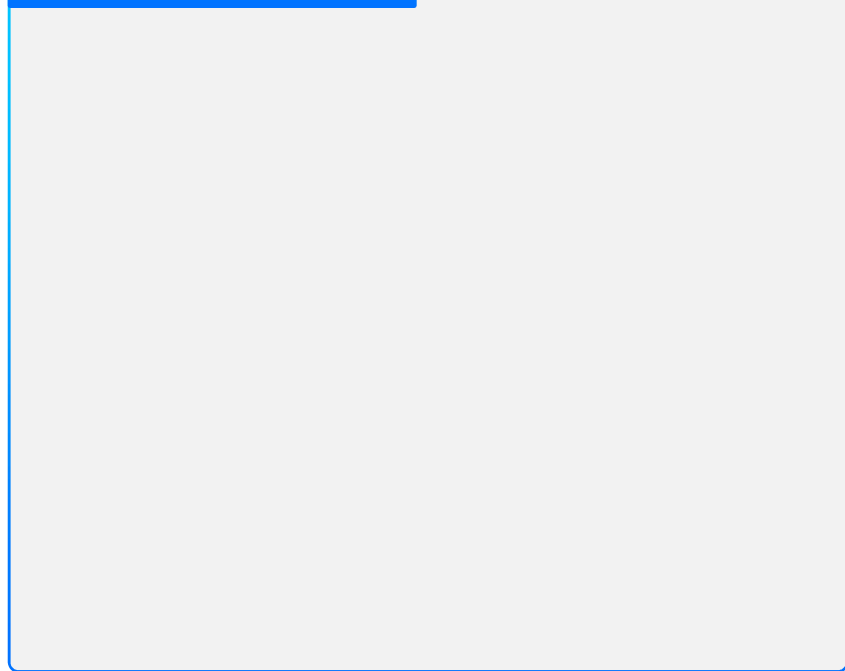
```
#!/bin/sh

salutation="Hello"
echo $salutation
echo "The program $0 is now running"
echo "The second parameter was $2"
echo "The first parameter was $1"
echo "The parameter list as a single string $*"
echo "The parameter list as a separate words $@"

exit 0
```

./try_variables.sh "CSBU108 is
awesome" right

Output:





EXERCISES

07



7. Exercises

Exercise: 01

List the contents of your `/etc` directory, including the ownership and permissions, and redirect the output to a file called `report_<studentID>.txt` within your home directory.

Exercise: 02

Sort the contents of the `report_<studentID>.txt` file from your current directory and append it to the end of a new file named `report-sorted.txt`.



7. Exercises

Exercise: 03

Count the number of words within the `report_<studentID>.txt` file and *append* the output to the end of a file `field2.txt` in your home directory. You will need to use both input and output redirection.

Exercise: 04

Using the previously created `report_<studentID>.txt` file, count the number of characters of the last 9 lines.